

Week 2: ML Foundations

Table of Contents

- [Core Idea First, No AI](#)
- [AI vs ML vs Analytics vs Rules](#)
- [The ML Loop](#)
- [Supervised vs Unsupervised Learning](#)
- [Generalization vs Overfitting-why-models-that-memorize-fail](#)
- [ML Task Types: Classification, Clustering, Ranking & Scoring](#)
- [Framing a Search Problem as an ML Task](#)

1. Core Idea First, No AI

Before reaching for machine learning, ask one question:

"Can I write down the rule?"

If you can express the decision as a clear, static condition **use the rule**. Rules are transparent, instant, free to run, and trivial to debug.

| Scenario | Rule works? | Why |

---|---|---

| Block orders from embargoed countries | Yes | The list is short, static, and legally defined |

| Detect spam emails | No | Spammers constantly change tactics; static keywords fail fast |

| Convert Celsius to Fahrenheit | Yes | It's a fixed formula: `F = C * 9/5 + 32` |

| Recommend the next video to watch | No | Depends on millions of user behaviors, context, and content features |

The core insight: ML earns its place when the decision surface is *too complex, too dynamic, or too high-dimensional* for a human to write down as explicit rules.

2. AI vs ML vs Analytics vs Rules

These terms are often used interchangeably, but they are *distinct layers* with different purposes:

`AI`

Artificial Intelligence (AI)	
"Machines that exhibit intelligent behavior"	

`ML`

Machine Learning (ML)	
"Systems that learn patterns from"	

data without being explicitly
 programmed for each case"

Deep Learning (DL)
 "Neural networks with many
 layers (LLMs live here)"

The Four Approaches Compared

Approach	What it does	Input	Output	Example
Rules	Execute fixed human-written logic	Conditions	Deterministic action	"If cart > \$50, free shipping"
Analytics	Describe & summarize what happened	Historical data	Dashboards, reports, KPIs	"Bounce rate was 42% last month"
Machine Learning	Learn patterns from data to predict or decide	Labeled or unlabeled data	Predictions, scores, clusters	"This user has 73% probability of churning"
AI (broad)	Any system mimicking human intelligence	Varies	Varies	Chatbots, robotics, self-driving cars

When to use which?

Problem complexity	RULES	ANALYTICS	ML	DEEP LEARNING / LLMs
Few conditions	Summarize past	Patterns too		
Unstructured data	Static logic	Slice & dice	complex to	(text, images, audio)
Known edge cases	Dashboards	hand-code	Requires massive training data	
learning	"What happened?"	Needs to generalize to unseen data	Transfer	valuable

!IMPORTANT

Key takeaway: ML is NOT a replacement for rules or analytics. It's a tool you reach for when the pattern is too complex to hand-code and you have enough data for the model to learn it.

3. The ML Loop

Machine learning is *not* a one-shot process. It's an iterative loop:

`mermaid`

graph LR

```
A["1. Define the Problem"] --> B["2. Collect & Prepare Data"]
B --> C["3. Choose a Model"]
C --> D["4. Train the Model"]
D --> E["5. Evaluate Performance"]
E --> F{"Good enough?"}
F -- No --> G["6. Iterate: tune, get more data, try new features"]
G --> B
F -- Yes --> H["7. Deploy & Monitor"]
H --> I["8. Model degrades over time"]
I --> B
```

Each step explained:

| Step | What happens | Search example |

| --- | --- | --- |

| **1. Define the Problem** | Frame a clear question with a measurable target | "Which search results should rank higher for a given query?" |

| **2. Collect & Prepare Data** | Gather labeled examples, clean them, split into train/test | Collect query-click logs; label relevant vs. irrelevant results |

| **3. Choose a Model** | Pick an algorithm appropriate for the task | Logistic regression for binary relevance, or LambdaMART for ranking |

| **4. Train** | Feed training data to the model so it learns patterns | Model learns that clicks + dwell time correlate with relevance |

| **5. Evaluate** | Measure performance on *held-out* test data | Check precision@10, NDCG, or MRR on queries the model hasn't seen |

| **6. Iterate** | Improve features, tune hyperparameters, fix data issues | Add query-document similarity features; remove noisy labels |

| **7. Deploy & Monitor** | Put the model into production; watch for drift | Serve live rankings; track click-through rate daily |

| **8. Retrain** | Data changes over time; the model becomes stale | Retrain monthly on fresh query logs |

</p>

<p>[!TIP]

The ML loop is *continuous*. A model in production is never "done" because user behavior changes, content changes, and the world changes. This is called **concept drift**, and it's why monitoring and retraining are essential.

</p>

4. Supervised vs Unsupervised Learning

These are the two fundamental paradigms of ML:

Supervised Learning "Learning with a teacher"

You have **labeled data** each example comes with the correct answer.

`Input (Features)                      Label (Target)`

`ΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇ`

`ΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇ`

`[email text, sender, links]      spam / not spam`

`[query, document features]      relevant / not relevant`

`[house sqft, bedrooms, zip]      price ($425,000)`

The model learns the mapping: `f(features)      label`

| Task Type | Label Type | Examples |

| ---|---|---|

| **Classification** | Discrete category | Spam detection, image recognition, sentiment analysis |

| **Regression** | Continuous number | Price prediction, temperature forecasting, ad bid estimation |

Unsupervised Learning "Learning without a teacher"

You have **no labels** the model discovers hidden structure on its own.

`Input (Features)                      ???`

`ΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇΓöÇ`

`ΓöÇΓöÇΓöÇ`

`[user browsing behavior]      discovers 4 natural user segments`

`[search queries]      groups similar queries together`

`[product descriptions]      finds topic clusters`

| Task Type | What it finds | Examples |

| ---|---|---|

| **Clustering** | Natural groups in data | Customer segmentation, query grouping, anomaly detection |

| **Dimensionality Reduction** | Simplified representations | PCA for visualization, embeddings |

| **Association** | Items that co-occur | "Customers who bought X also bought Y" |

Side-by-side comparison

| Aspect | Supervised | Unsupervised |

| ---|---|---|

| **Data requirement** | Labeled examples (expensive to create) | Raw data (cheaper, more abundant) |

| **Goal** | Predict a known target | Discover hidden patterns |

| **Evaluation** | Clear metrics (accuracy, F1, RMSE) | Harder to evaluate (silhouette score, human judgment) |

| **Risk** | Needs high-quality labels | May find meaningless patterns |

| **Search example** | "Is this result relevant to the query?" | "What types of queries do users ask?" |

<blockquote>

<p>[!NOTE]

There's also **semi-supervised** learning (small amount of labeled + large amount of unlabeled data) and **reinforcement**

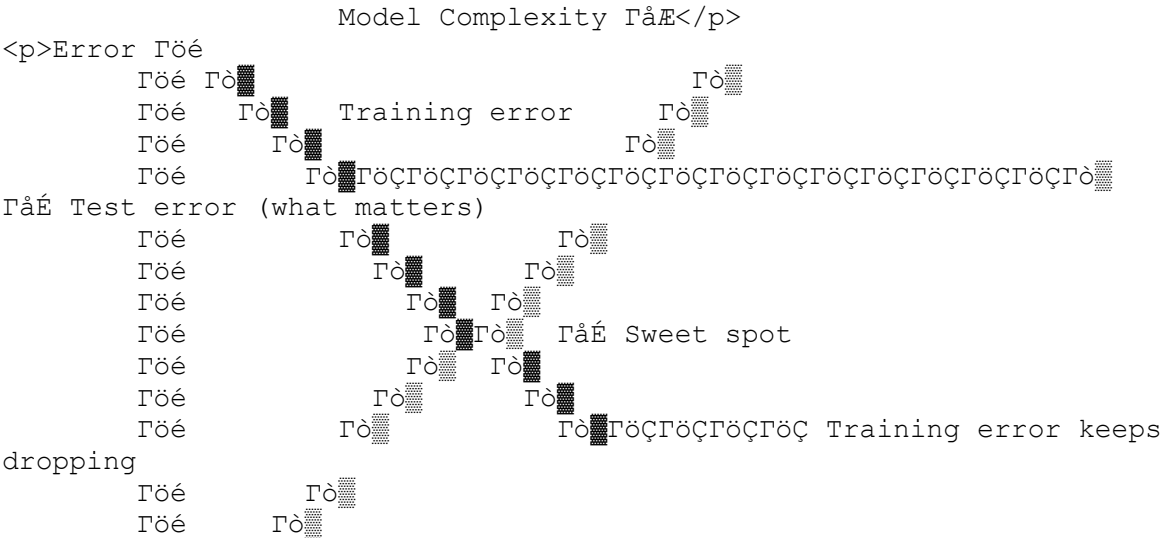
learning (agent learns through trial-and-error rewards). But supervised and unsupervised are the foundations.

5. Generalization vs Overfitting ¶ Why Models That Memorize Fail

This is perhaps the *most important* concept in all of ML.

The problem

```



UNDERFIT GOOD FIT OVERFIT

```

Three scenarios

	Underfitting	Good Fit	Overfitting
Training accuracy	Low	High	Very high (≈100%)
Test accuracy	Low (big gap from training)	High (close to training)	Low
The model	Too simple; misses patterns	Captures real patterns	Memorized noise in training data
Analogy	A student who didn't study	A student who understood concepts	A student who memorized answers but can't solve new problems

Why does overfitting happen?

- Too little data ¶ The model doesn't see enough variety
- Too complex a model ¶ The model has more capacity than the data warrants
- Training too long ¶ The model starts fitting noise after learning signal
- Noisy labels ¶ Errors in the training data are treated as truth

Why it matters for real-world systems

```

<blockquote>
<p>[!CAUTION]
<strong>A model that memorizes its training data will fail
catastrophically on new, real-world inputs.</strong> This is why we
always evaluate on held-out test data that the model has <em>never</em>
seen during training.</p>
</blockquote>
<p><strong>Search example:</strong> Imagine training a relevance model
only on queries from January. If user behavior shifts in February
(seasonal trends, news events, new products), the overfitted model will
make bad predictions because it memorized January's patterns rather than
learning the <em>underlying</em> concept of relevance.</p>
<h3>Defenses against overfitting</h3>
<p>| Technique | How it helps |
|---|---|
| <strong>Train/Test Split</strong> | Evaluate on data the model hasn't
seen |
| <strong>Cross-Validation</strong> | Multiple train/test splits for more
robust estimates |
| <strong>Regularization</strong> | Penalize overly complex models (L1,
L2, dropout) |
| <strong>More data</strong> | Harder to memorize a larger, more diverse
dataset |
| <strong>Early stopping</strong> | Stop training when test performance
starts degrading |
| <strong>Simpler model</strong> | Sometimes a linear model beats a deep
neural network |</p>
<hr />
<h2>6. ML Task Types</h2>
<p>Every ML problem maps to one of a few core task types. For search and
SEO applications, these four are the most relevant:</p>
<h3>Classification</h3>
<blockquote>
<p><strong>Question:</strong> "Which category does this belong to?"</p>
</blockquote>
<ul>
<li><strong>Output:</strong> A discrete label (binary or multi-
class)</li>
<li><strong>Examples:</strong></li>
<li>Is this page relevant or irrelevant to the query? ¶ Binary
classification</li>
<li>Is user intent navigational, informational, or transactional? ¶
Multi-class classification</li>
<li>Is this content spam? ¶ Binary classification</li>
<li><strong>Metrics:</strong> Accuracy, Precision, Recall, F1-score, AUC-
ROC</li>
</ul>
<h3>Clustering</h3>
<blockquote>
<p><strong>Question:</strong> "What natural groups exist in this
data?"</p>
</blockquote>
<ul>

```

```

<li><strong>Output:</strong> Group assignments (no predefined
labels)</li>
<li><strong>Examples:</strong></li>
<li>Group search queries by user intent patterns</li>
<li>Segment website visitors by behavior</li>
<li>Identify content topic clusters for SEO strategy</li>
<li><strong>Metrics:</strong> Silhouette score, within-cluster sum of
squares, human evaluation</li>
</ul>
<h3>Ranking</h3>
<blockquote>
<p><strong>Question:</strong> "In what order should these items
appear?"</p>
</blockquote>
<ul>
<li><strong>Output:</strong> An ordered list from most to least
relevant/important</li>
<li><strong>Examples:</strong></li>
<li>Rank search results for a query</li>
<li>Rank pages for internal linking priority</li>
<li>Rank keywords by opportunity score</li>
<li><strong>Metrics:</strong> NDCG (Normalized Discounted Cumulative
Gain), MRR (Mean Reciprocal Rank), MAP (Mean Average Precision)</li>
</ul>
<h3>Scoring</h3>
<blockquote>
<p><strong>Question:</strong> "How strong/likely/important is this?"</p>
</blockquote>
<ul>
<li><strong>Output:</strong> A continuous numerical score</li>
<li><strong>Examples:</strong></li>
<li>Predict click-through rate for a search result</li>
<li>Score page authority (0FÇô100)</li>
<li>Predict probability of conversion after landing</li>
<li><strong>Metrics:</strong> RMSE, MAE, RT2, calibration plots</li>
</ul>
<h3>How they connect</h3>
<p>``mermaid
graph TD
    subgraph Supervised
        CL["Classification<br/>'Which category?'" ]
        RK["Ranking<br/>'What order?'" ]
        SC["Scoring<br/>'How much/likely?'" ]
    end
    subgraph Unsupervised
        CLU["Clustering<br/>'What groups exist?'" ]
    end
    end</p>
<pre><code>SC --&gt;|"threshold the score"| CL
SC --&gt;|"sort by score"| RK
CL --&gt;|"predict scores per class"| SC
CLU --&gt;|"label clusters, then classify"| CL
</code></pre>
<p>``</p>
</blockquote>

```


| **Input Features** | Word count, domain authority, number of backlinks, keyword density, content freshness, page speed score, internal link count |

| **Success Metric** | **F1-score** ≈ 0.70 on held-out test set (balanced between precision and recall because both false positives and false negatives are costly) |

| **Why not a rule?** | There's no single threshold (e.g., "word count > 2000 ") that reliably predicts ranking. It depends on a complex interaction of 200+ ranking factors that vary by query intent, competition, and domain. |

Another example: "How should I prioritize my keyword opportunities?"

| Component | Value |

---|---

| **Search Question** | "Given a set of keywords I could target, which ones should I write about first?" |

| **ML Task Type** | **Scoring** $\rightarrow$ then **Ranking** (score each keyword, then sort) |

| **Target** | `opportunity_score` $\rightarrow$ continuous value (0-100) combining search volume, difficulty, and business relevance |

| **Input Features** | Monthly search volume, keyword difficulty, current rank (if any), CPC, content gap analysis, topical authority score |

| **Success Metric** | **NDCG@10** ≈ 0.75 $\rightarrow$ the top 10 recommended keywords should closely match what an expert SEO strategist would pick |

| **Why not a rule?** | Simple rules like "pick highest volume + lowest difficulty" ignore business relevance, topical clustering, and competitive dynamics. The trade-offs are multi-dimensional. |

Key Takeaways

[!IMPORTANT]

What you should leave with:

- ML $\neq$ AI $\neq$ Analytics $\neq$ Rules** $\rightarrow$ each has its lane; use the simplest tool that works
- The ML loop is iterative** $\rightarrow$ define $\rightarrow$ collect $\rightarrow$ train $\rightarrow$ evaluate $\rightarrow$ improve $\rightarrow$ deploy $\rightarrow$ monitor $\rightarrow$ repeat
- Supervised learning needs labels; unsupervised learning finds structure** $\rightarrow$ your task type determines which you need
- Overfitting is the #1 enemy** $\rightarrow$ a model that memorizes training data will fail on real-world data; always evaluate on held-out test sets
- Every ML task has a type, a target, and a metric** $\rightarrow$ if you can't name these three things, you don't have an ML task yet
- Your capstone starts here** $\rightarrow$ frame your search question as a real ML task with a defensible success metric

References & Further Reading

```
<ul>
<li><strong>Session recording:</strong> <a
href="https://www.youtube.com/@flyrank">FlyRank YouTube Channel</a></li>
<li><strong>Video material:</strong> <a
href="https://youtu.be/aTxLqzQ5Isg">Week 2 Session TÇö Applied Search
Intelligence</a></li>
<li>Google Machine Learning Crash Course: <a
href="https://developers.google.com/machine-learning/crash-
course">developers.google.com/machine-learning/crash-course</a></li>
<li>Scikit-learn documentation on <a href="https://scikit-
learn.org/stable/modules/model_evaluation.html">model evaluation
metrics</a></li>
</ul>
```